

COMPUTER VISION LAB MANUAL
Course Code: SCSB2752

Course Outcomes

- CO1: Apply image preprocessing techniques using OpenCV.
- CO2: Implement image segmentation and feature extraction algorithms.
- CO3: Develop object, face, and vehicle detection systems.
- CO4: Apply OCR and scene understanding techniques.
- CO5: Build deep-learning-based computer vision applications.

List of Experiments

1. Basic Image Handling and Processing using OpenCV
2. Image 2D to 3D Conversion
3. Motion Detection and Tracking using OpenCV
4. Image Captioning using Vision Transformer (ViT-GPT2)
5. Vehicle Detection using Haar Cascade Classifier
6. Contour-Based Image Segmentation
7. Region-Based Image Segmentation using Watershed Algorithm
8. Edge Detection using Sobel and Canny Operators
9. Shape Detection using Hough Transform
10. Face Detection using Haar Cascade Classifier
11. Scene Text Detection using OCR (Tesseract)
12. Road Lane Detection for Autonomous Vehicles
13. Emotion Recognition using DeepFace
14. Object Detection using YOLO
15. Vehicle Counting in Images and Videos
16. QR Code Detection and Decoding

Experiment 1: Basic Image Handling and Processing using OpenCV

Aim

To study and implement basic image handling and processing using opencv.

Algorithm

1. Load the input image.
2. Convert the image to grayscale.
3. Resize the image to a fixed dimension.
4. Apply Gaussian blur to reduce noise.
5. Perform edge detection using the Canny operator.
6. Draw geometric shapes (rectangle, line) on the image.
7. Display the processed outputs.

Program

```
import cv2

from google.colab.patches import cv2_imshow

# Load image
image = cv2.imread('set image path')
cv2_imshow(image)

# Convert to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2_imshow(gray_image)

# Resize image
resized_image = cv2.resize(image, (200, 200))
cv2_imshow(resized_image)

# Apply Gaussian blur
blurred_image = cv2.GaussianBlur(image, (15, 15), 0)
cv2_imshow(blurred_image)

# Edge detection
```

```
edges = cv2.Canny(gray_image, 100, 200)
cv2_imshow(edges)

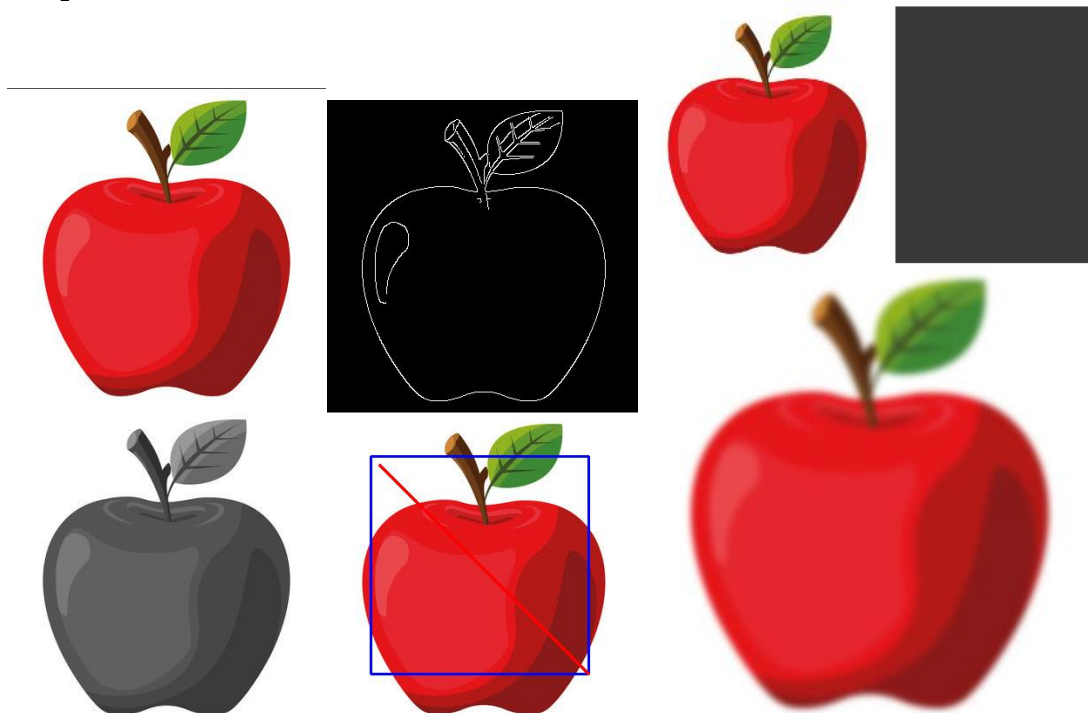
# Draw rectangle and line
cv2.rectangle(image, (50, 50), (300, 300), (255, 0, 0), 2)
cv2.line(image, (60, 60), (300, 300), (0, 0, 255), 2)
cv2_imshow(image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input:



Output:



Result

Thus, the experiment was successfully executed.

Experiment 2: Image 2D to 3D Conversion

Aim

To study and implement image 2D to 3D conversion using depth maps and pixel shifting techniques.

Algorithm

- Load the base image and its corresponding depth image.
- Convert the base image to RGBA format and the depth image to grayscale.
- Normalize depth values to calculate pixel shifts.
- Shift pixels horizontally based on depth values.
- Construct the new image with shifted pixels.
- Display the resulting pseudo-3D image.

Program

```
from PIL import Image

import numpy as np

def shift_image(img, depth_img, shift_amount=10):

    # Ensure base image has alpha
    img = img.convert("RGBA")

    data = np.array(img)

    # Ensure depth image is grayscale (for single value)
    depth_img = depth_img.convert("L")

    depth_data = np.array(depth_img)

    deltas = ((depth_data / 255.0) * float(shift_amount)).astype(int)

    # This creates the transparent resulting image.
    # For now, we're dealing with pixel data.
    shifted_data = np.zeros_like(data)

    height, width, _ = data.shape

    for y, row in enumerate(deltas):

        for x, dx in enumerate(row):

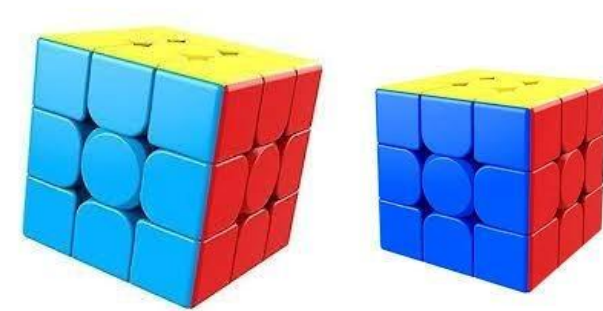
            if x + dx < width and x + dx >= 0:

                shifted_data[y, x + dx] = data[y, x]
```

```
# Convert the pixel data to an image.  
shifted_image = Image.fromarray(shifted_data.astype(np.uint8))  
return shifted_image  
  
img = Image.open("C:\\Users\\student\\Desktop\\cube1.jpeg")  
depth_img = Image.open("C:\\Users\\student\\Desktop\\cube3.jpeg")  
shifted_img = shift_image(img, depth_img, shift_amount=10)  
shifted_img.show()
```

Output

Input Images:



Output Image:



Result

Thus, the experiment was successfully executed.

Experiment 3: Motion Detection and Tracking using OpenCV

Aim

To study and implement motion detection and tracking in video sequences using OpenCV.

Algorithm

1. Load the input video.
2. Initialize a background subtractor.
3. Read frames sequentially.
4. Apply background subtraction to detect foreground objects.
5. Use contour detection to identify moving objects.
6. Track the centroid of each moving object.
7. Display the annotated video frames.

Program

```
import cv2

def detect_moving_objects(video_path):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print("Error: Couldn't open the video file.")
        return

    bg_subtractor = cv2.createBackgroundSubtractorMOG2()

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        fg_mask = cv2.medianBlur(bg_subtractor.apply(frame), 5)
        contours, _ = cv2.findContours(fg_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        for contour in contours:
            if cv2.contourArea(contour) > 100:
                M = cv2.moments(contour)
                if M["m00"] != 0:
                    cx = int(M["m10"] / M["m00"])
                    cy = int(M["m01"] / M["m00"])
                    # Draw a dot (small filled circle)
```

```
cv2.circle(frame, (cx, cy), 5, (255, 0, 0), 5)
cv2.imshow('Moving Object Detection', frame)
if cv2.waitKey(45) & 0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()
video_path = 'input.mp4'
detect_moving_objects(video_path)
```

Output:



Result

Thus, motion detection and tracking were successfully implemented using OpenCV.

Experiment 4: Image Captioning using Vision Transformer (ViT-GPT2)

Aim

To study and implement image captioning using Vision Transformer (ViT) for feature extraction and GPT-2 for natural language generation.

Algorithm

- Load the pre-trained ViT-GPT2 model.
- Preprocess the input image (resize, normalize).

- Extract features using the Vision Transformer.
- Generate captions using GPT-2 decoder.
- Display the predicted caption.

Program

```
from transformers import VisionEncoderDecoderModel, ViTFeatureExtractor,
AutoTokenizer

import torch

from PIL import Image

import warnings

warnings.filterwarnings('ignore')

model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-gpt2-image-
captioning")

feature_extractor = ViTFeatureExtractor.from_pretrained("nlpconnect/vit-gpt2-image-
captioning")

tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-image-captioning")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.to(device)

max_length = 16

num_beams = 4

gen_kwargs = {"max_length": max_length, "num_beams": num_beams}

def predict_step(image_paths):
    images = []

    for image_path in image_paths:
        i_image = Image.open(image_path)

        if i_image.mode != "RGB":
            i_image = i_image.convert(mode="RGB")

        images.append(i_image)

    pixel_values = feature_extractor(images=images, return_tensors="pt").pixel_values

    pixel_values = pixel_values.to(device)

    output_ids = model.generate(pixel_values, **gen_kwargs)

    preds = tokenizer.batch_decode(output_ids, skip_special_tokens=True)
```



```
preds = [pred.strip() for pred in preds]
return preds
predict_step(["C:\\Users\\student\\Downloads\\ss.jpg"])
```

Output

['young children playing with a soccer ball']
Download image below :



Result

Thus, image captioning was successfully implemented using Vision Transformer (ViT) and GPT-2, generating meaningful textual descriptions for input images.

Experiment 5: Vehicle Detection using Haar Cascade Classifier

Aim

To study and implement vehicle detection using haar cascade classifier.

Algorithm

1. Load the input image or video.
2. Convert the image to grayscale.
3. Apply preprocessing (Gaussian blur, dilation, morphological operations).
4. Load the Haar Cascade XML classifier for vehicles.
5. Detect vehicles using detectMultiScale().
6. Draw bounding boxes around detected vehicles.
7. Display the annotated image/video.

Program

```
from PIL import Image
import cv2
import numpy as np
import requests
image_url = 'sample.jpg'
image = Image.open(image_url)
image = image.resize((450, 250))
image.show()
cv2.waitKey(0)
cv2.destroyAllWindows()
image_arr = np.array(image)
grey = cv2.cvtColor(image_arr, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(grey, (5, 5), 0)
dilated = cv2.dilate(blur, np.ones((3, 3)))
dilated = cv2.dilate(blur, np.ones((3, 3)))
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2, 2))
closing = cv2.morphologyEx(dilated, cv2.MORPH_CLOSE, kernel)
car_cascade_src = "vehicle.xml"
car_cascade = cv2.CascadeClassifier(car_cascade_src)
cars = car_cascade.detectMultiScale(closing, 1.1, 1)
cnt = 0

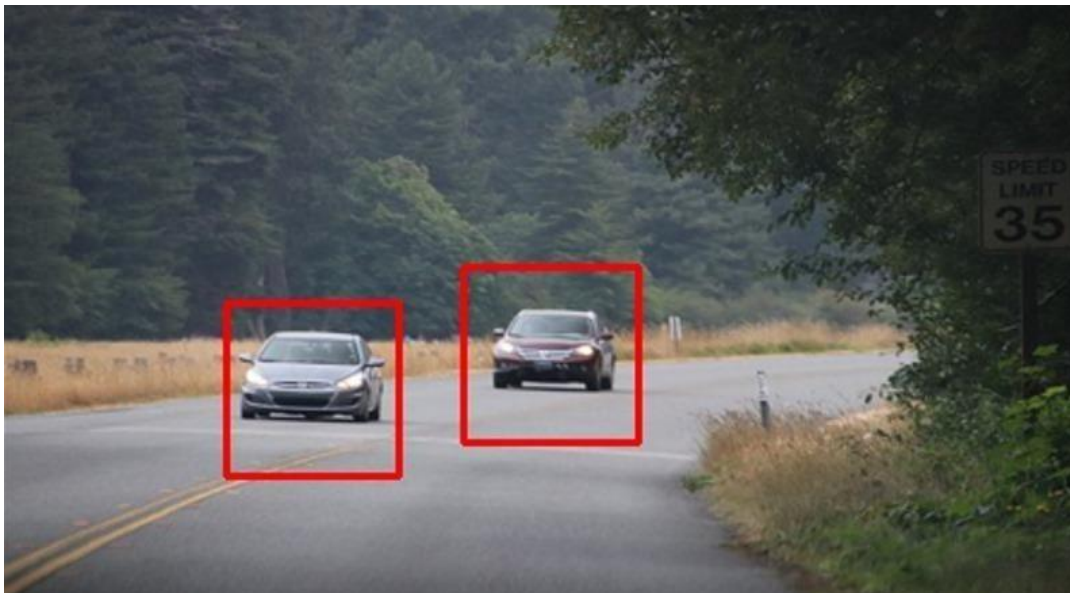
for (x, y, w, h) in cars:
    cv2.rectangle(image_arr, (x, y), (x + w, y + h), (255, 0, 0), 2)
    cnt += 1

annotated_image = Image.fromarray(image_arr)
annotated_image.show()
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input:



Output:



Result

Thus, vehicle detection was successfully implemented using Haar Cascade Classifier in OpenCV.

Experiment 6: Contour-Based Image Segmentation

Aim

To study and implement contour-based image segmentation.

Algorithm

- Load the input image.
- Convert the image to grayscale.
- Apply Gaussian blur to reduce noise.
- Perform edge detection using the Canny operator.
- Detect contours using cv2.findContours().
- Draw contours on the original image.
- Display the results.

Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Load the image

image = cv2.imread('path_to_your_image.jpg')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply GaussianBlur to reduce noise and improve contour detection

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Perform edge detection

edged = cv2.Canny(blurred, 50, 150)

# Find contours

contours, _ = cv2.findContours(edged, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# Draw contours on the original image

contour_image = image.copy()

cv2.drawContours(contour_image, contours, -1, (0, 255, 0), 2)

# Display the results

plt.figure(figsize=(10, 10))
```

```

plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.subplot(1, 3, 2)
plt.title('Edge Detection')
plt.imshow(edged, cmap='gray')
plt.subplot(1, 3, 3)
plt.title('Contours')
plt.imshow(cv2.cvtColor(contour_image, cv2.COLOR_BGR2RGB))
plt.show()

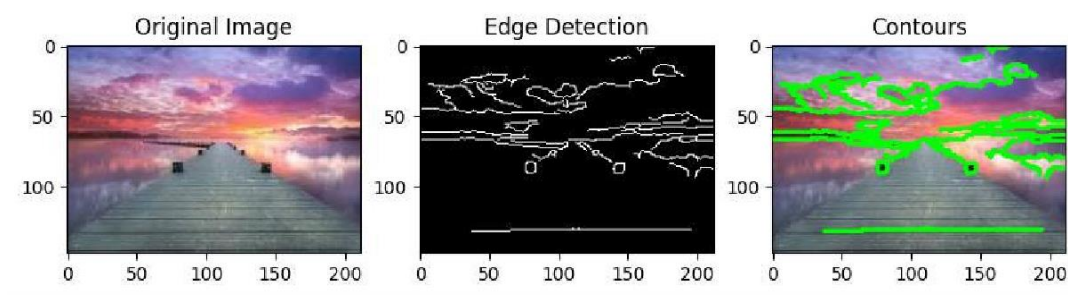
```

Output

Input:



Output:



Result

Thus, contour-based image segmentation was successfully implemented using OpenCV.

Experiment 7: Region-Based Image Segmentation using Watershed Algorithm

Aim

To study and implement region-based image segmentation using watershed algorithm.

Algorithm

1. Load the input image and convert to grayscale.
2. Apply edge detection (Canny).
3. Fill regions to separate objects.
4. Compute elevation map using Sobel filter.
5. Create markers for watershed segmentation.
6. Apply watershed algorithm.
7. Display segmented regions.

Program

```
import numpy as np

import matplotlib.pyplot as plt

from skimage.feature import canny

from skimage import data, segmentation, morphology, filters

from skimage.color import rgb2gray, label2rgb

import scipy.ndimage as nd

plt.rcParams["figure.figsize"] = (12, 8)

%matplotlib inline

# Load image and convert to grayscale

rocket = data.rocket()

rocket_wh = rgb2gray(rocket)

# Apply Canny edge detection

edges = canny(rocket_wh)

plt.imshow(edges, interpolation='gaussian')

plt.title('Canny detector')

plt.show()

# Fill regions to perform edge segmentation

fill_im = nd.binary_fill_holes(edges)
```

```
plt.imshow(fill_im)
plt.title('Region Filling')
plt.show()

# Compute the elevation map using the Sobel filter
elevation_map = filters.sobel(rocket_wh)
plt.imshow(elevation_map)
plt.title('Elevation Map')
plt.show()

# Create markers for watershed
markers = np.zeros_like(rocket_wh)
markers[rocket_wh < 0.1171875] = 1 # 30/255
markers[rocket_wh > 0.5859375] = 2 # 150/255
plt.imshow(markers)
plt.title('Markers')
plt.show()

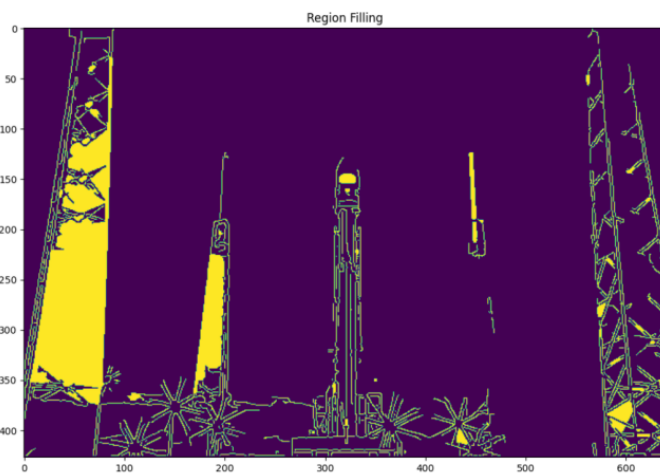
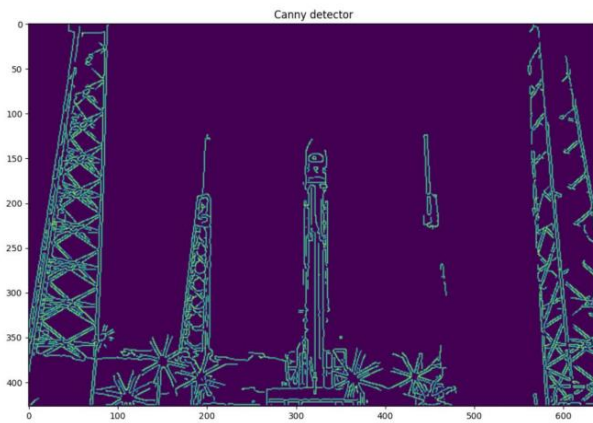
# Perform watershed segmentation
segments = segmentation.watershed(elevation_map, markers)
plt.imshow(segments)
plt.title('Watershed Segmentation')
plt.show()

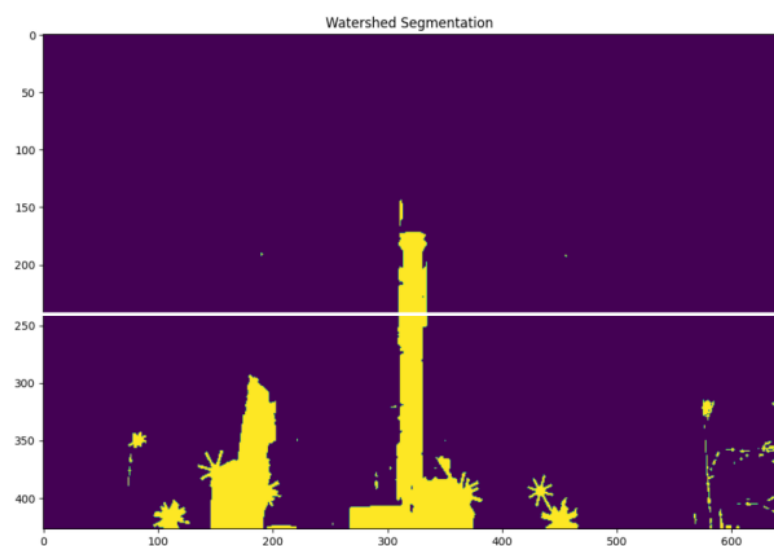
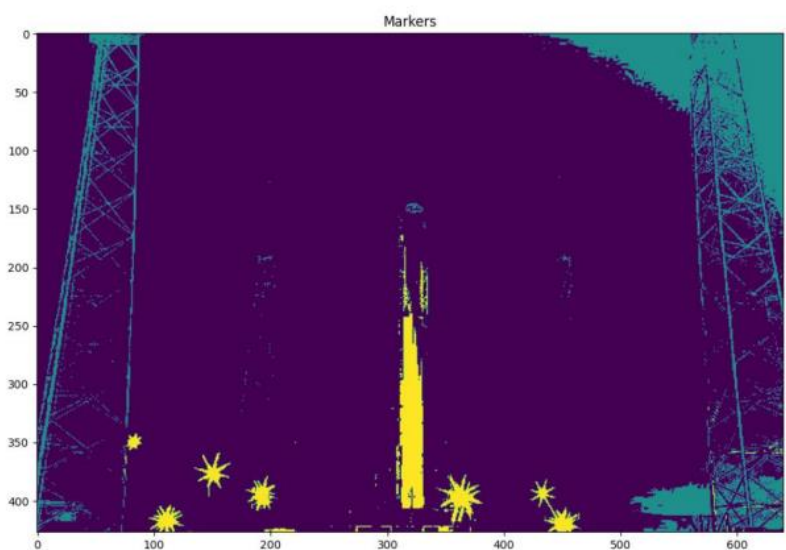
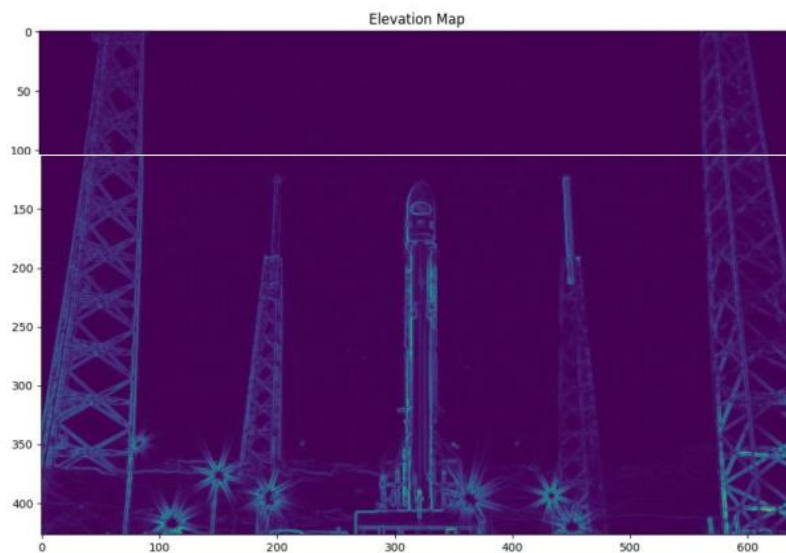
# Fill holes in the segmented image
segments_filled = nd.binary_fill_holes(segments - 1)
label_rock, _ = nd.label(segments_filled)

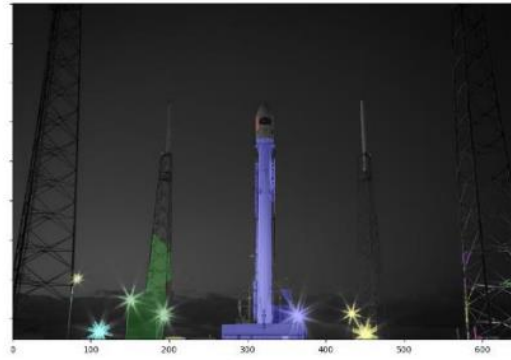
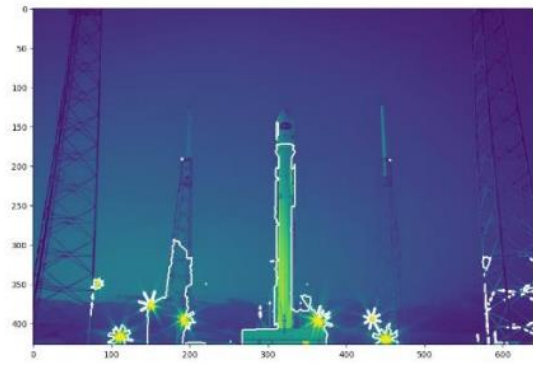
# Overlay image with different labels
image_label_overlay = label2rgb(label_rock, image=rocket_wh)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(24, 16), sharey=True)
ax1.imshow(rocket_wh)
ax1.contour(segments_filled, [0.8], linewidths=1.8, colors='w')
ax2.imshow(image_label_overlay)
```

```
plt.show()
```

Output







Result

Thus, region-based segmentation using watershed was successfully implemented.